

Entregable 2: Segmentación de imagen mediante arquitectura U-Net

Aprendizaje Profundo: Ciencia de Datos 2025/26

Carlos Mendes, Pablo Méndez, Pablo Múgica, Iván Nieto

Índice del informe:

1. Versión inicial de la pipeline de segmentación.....	2
1.1. Modelo.....	2
1. Camino Descendente (Encoder).....	3
2. Cuello de Botella.....	3
3. Camino Ascendente (Decoder).....	3
4. Capa de Salida (OutConv).....	4
1.2. Entrenamiento inicial.....	4
Función de pérdida.....	4
Optimizador y política de evolución del factor de aprendizaje.....	4
Métricas de rendimiento.....	4
1.3. Resultados modelo inicial.....	5
2. Propuestas de mejora.....	7
2.1. Modificación de la carga y transformación de datos.....	7
2.2. Modificación de la arquitectura del modelo.....	8
Técnicas de regularización.....	8
Aumento del número de capas.....	8
Agrupamiento de las clases en postprocesado.....	8
2.3. Modificación de la función de coste.....	9
2.4. Modificaciones del algoritmo de entrenamiento (o del rendimiento del modelo).....	9
Learning rate scheduler.....	9
2.5. Otras métricas de rendimiento.....	9
2.6. Resultados modelo mejorado.....	9

1. Versión inicial de la pipeline de segmentación

La versión inicial consiste en un modelo simple base construido mediante la arquitectura U-Net con la capacidad de llevar a cabo un entrenamiento estándar, aplicando métricas de rendimiento aceptables. El modelo resultante en este apartado no está optimizado ni está creado en búsqueda del mejor rendimiento, se trata de un modelo inicial para observar un aprendizaje básico con el paso de las épocas.

1.1. Modelo

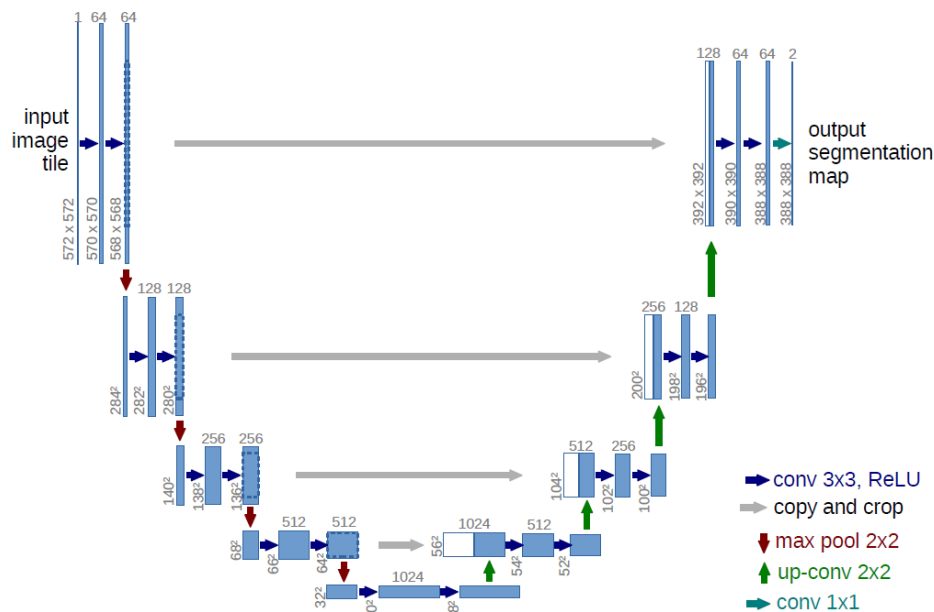
El modelo sigue la estructura compacta de la arquitectura U-Net. Con ella, el objetivo será tener salidas con el mismo tamaño que la entrada, en donde devolveremos una máscara con la clase predicha para cada pixel. Para ello, cada imagen en el conjunto de entrenamiento va acompañada de su correspondiente máscara de píxeles donde cada píxel tiene una predicción de clase.

La entrada tiene una forma $B \times C \times H \times W$ (Lote x Canales x Altura x Ancho). En el modelo, la entrada inicial tiene $in_channels = 3$ ya que son imágenes a color en formato RGB.

Durante el proceso, la imagen es codificada a una representación de baja resolución (Encoder) y luego decodificada de nuevo a la resolución original (Decoder).

La salida se compone de un tensor de *logits* con forma $B \times num_clases \times H \times W$, con igual altura y ancho que la entrada, pero con un número de canales igual al número de clases del dataset.

La estructura del modelo sigue la siguiente estructura, la cual se irá describiendo bloque a bloque posteriormente:



1. Camino Descendente (Encoder)

El encoder se encarga de capturar el contexto y las características de más nivel, reduciendo progresivamente la resolución espacial mientras aumenta la profundidad (número de canales). Se conforma del bloque de Doble Convolución y el bloque Down.

- Bloque de Doble Convolución (*DoubleConv*)

Este bloque es la primera estructura del modelo, la cual se compone de una doble convolución 3x3, donde se aplica la función de activación no lineal *ReLU* tras cada una de ellas.

El propósito de esta estructura es la extracción de características de la imagen de manera profunda, manteniendo la misma resolución espacial, pero aumentando el número de canales desde 3 hasta 64. La primera convolución es la encargada de realizar el cambio de profundidad, mientras que la segunda mantiene la profundidad y se enfoca en refinar las características.

- Bloque Down (*Down*)

El bloque Down se conforma de una capa de *MaxPooling* 2x2 seguida de un bloque *DoubleConv*. La capa de *MaxPooling* reduce la altura y el ancho de la imagen a la mitad sucesivamente, mientras que el *DoubleConv* duplica el número de canales en cada aplicación. Por ello, se reduce a la mitad la resolución espacial mientras se duplica la profundidad.

El bloque Down se aplica cuatro veces de manera consecutiva tras el bloque *DoubleConv*, la resolución espacial disminuye hasta 9 x12, mientras que el número de canales aumenta hasta 1024.

2. Cuello de Botella

El cuello de botella es la parte más profunda de la red, conectando el *Encoder* y el *Decoder*.

Esta estructura se conforma de un bloque *DoubleConv* que procesa la representación de características más abstractas y de menor resolución antes de empezar el camino ascendente.

3. Camino Ascendente (Decoder)

El camino ascendente (decoder) se encarga de reconstruir la resolución espacial original combinando la información obtenida del encoder con las conexiones de salto (skip connections). Se conforma de un único bloque denominado *Up*.

- Bloque Up (*Up*):

El bloque *Up* se compone de una primera capa de deconvolución 2x2 que aumenta la resolución espacial (altura y anchura) al doble y reduce a la mitad el número de canales del tensor principal. A continuación, el resultado se concatena con el mapa de características de la

capa correspondiente del encoder mediante una conexión de salto (skip connection). Estas conexiones sirven para que el modelo tenga más “rutas cortas” sobre las que calcular el gradiente, lo que reduce el problema del gradiente desvaneciente. Por último, presenta un bloque de doble convolución (*DoubleConv*) para refinar la información fusionada. Este bloque se aplica el mismo número de veces que el bloque *Down* para recuperar la resolución original de la imagen (tensor) y el mismo número de canales.

4. Capa de Salida (*OutConv*)

La capa de salida consiste en una capa de convolución 1x1 que mapea el tensor de características finales (con 64 canales y la resolución original) a un tensor con *num_clases* canales. Cada píxel tiene un vector de logits, donde cada uno corresponde a una posible clase del dataset.

Para obtener la predicción final de cada píxel, se aplica argmax sobre el vector de logits, seleccionando la clase con mayor valor. No es necesario el uso de softmax en este caso, ya que al ser una función monótona creciente, la clase con mayor logit será también la clase con mayor probabilidad, por lo que la elección de la clase predicha será la misma en ambos casos.

1.2. Entrenamiento inicial

Función de pérdida

Como función de pérdida se ha elegido usar la función *CrossEntropyLoss* para múltiples clases, ya que nuestro dataset contiene 61 clases en total y por ser una función de pérdida estándar. Esta función calcula la pérdida píxel a píxel, comparando el logit predicho para cada píxel con su etiqueta real:

$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

Esta función penaliza al modelo cuando asigna un valor bajo (baja confianza) al logit correspondiente a la clase correcta y recompensa cuando ese logit es alto, guiando el aprendizaje hacia predicciones más precisas.

Optimizador y política de evolución del factor de aprendizaje

Respecto al optimizador, inicialmente usaremos el optimizador Adam con un learning rate fijado a 0.001. El objetivo en este apartado es que el modelo sea capaz de aprender de manera básica, por lo que no haremos comparaciones entre optimizadores ni ajuste de parámetros de los mismos.

Métricas de rendimiento

La métrica de rendimiento elegida es el accuracy, que para una imagen se calcula como el número de píxeles clasificados correctamente entre el total de píxeles de la imagen. Sin embargo veremos que estamos en un problema de clases desbalanceadas, ya que tenemos objetos con poca cantidad de píxeles respecto al fondo (background), por lo que un modelo que solo aprenda a clasificar el fondo tendrá buen valor en esta métrica, y será necesario añadir otras para asegurar buen rendimiento total.

1.3. Resultados modelo inicial

(Fasel.ipynb)

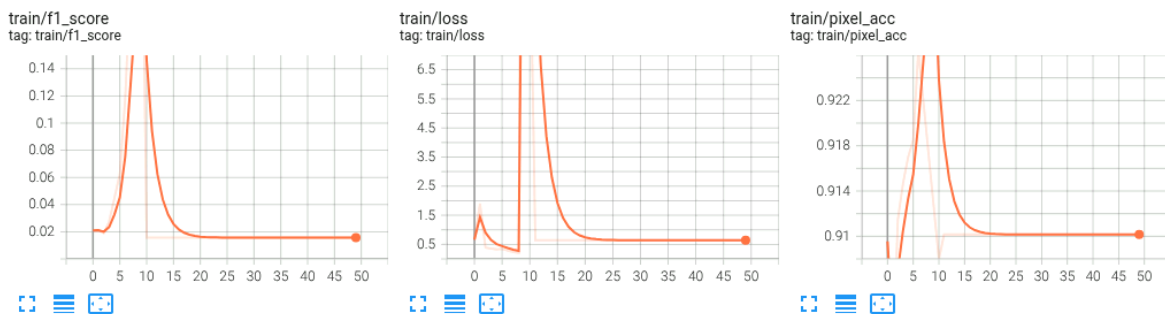
En este apartado se presentan finalmente los resultados obtenidos por la versión inicial del modelo U-Net. Para el entrenamiento del modelo se ha seguido la lógica del SGD (Stochastic Gradient Descent), donde se realizaron 50 épocas con $batch_size = 4$ utilizando el conjunto de datos de train dividido en conjuntos de entrenamiento y validación. Durante cada época se registra el valor de la función de pérdida y del accuracy para ambos conjuntos.

Adicionalmente, calcularemos el IoU (Intersection over Union) para cada clase del conjunto de test. Con ello, observaremos que tan bien aprende para predecir cada clase, ya que un valor grande de accuracy puede deberse a predecir correctamente el fondo pero no las clases.

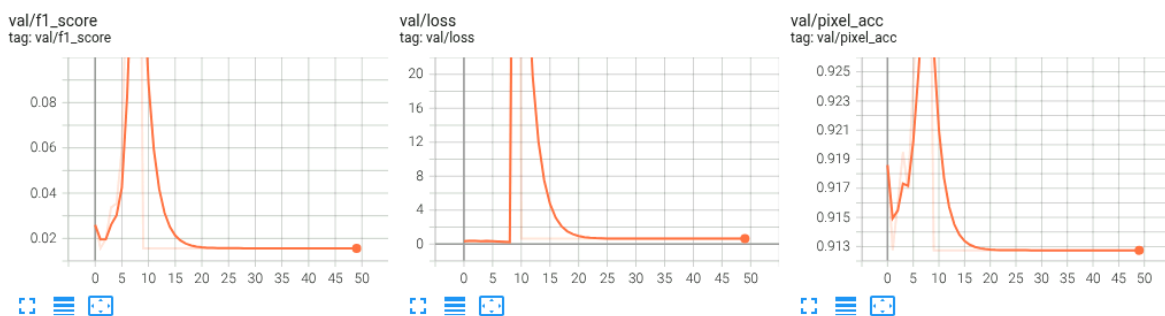
Como se ha comentado anteriormente, el entrenamiento se realiza con el optimizador Adam con el learning rate fijo en 0.001, y sin el uso de schedulers. A continuación se muestran los resultados obtenidos:

Evolución de la pérdida y accuracy

train

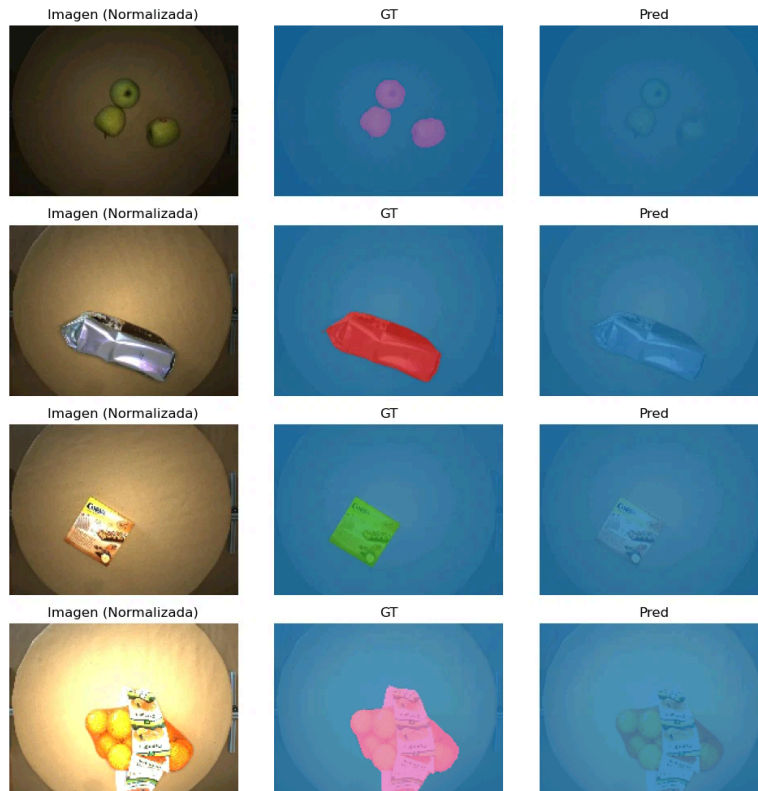


val



Esto parece un caso de gradiente explosivo, aunque luego las métricas vuelven a disminuir. Vemos que el *loss* se dispara entre las primeras *epochs*, puede que a causa del tamaño del *learning rate*, lo cual arruina el entrenamiento del modelo.

Predicciones reales

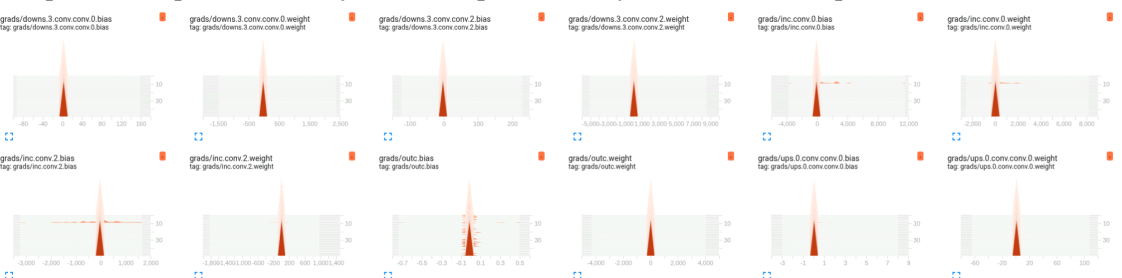


Vemos que el modelo solo está prediciendo fondo. Por esa razón el modelo tiene tan buen accuracy en test pero bajo F1 o mIoU, por lo que este modelo no sirve para nada.

Series temporales de parámetros e histogramas de gradientes



Vemos que las distribuciones de los parámetros son en general bastante estables a partir de la epoch 10, lo que indica que el modelo ya no está aprendiendo y ha alcanzado un punto estable.



Además vemos que los gradientes están centrados en 0 pero con mucha variabilidad, lo que puede explicar el pico extremo en las métricas.

Rendimiento en el conjunto de test

Todas las gráficas y resultados indican que el modelo ha únicamente aprendido a clasificar todo como fondo. Aún así, se realizaron predicciones para un conjunto de test, donde podemos verificar todas las hipótesis que tenemos. Los resultados son los siguientes:

- **Test Loss:** 1.4497
- **Clase 0 - Acc:** 1.0000 | **IoU:** 0.7848
- **Clase 1-60 - Acc:** 0.0000 | **IoU:** 0.0000

Con estos resultados podemos validar la información que teníamos a partir del conjunto de validación: el modelo predice cada píxel de cada imagen como fondo. Podemos ver que la clase 0 tiene un accuracy de 1, mientras que el resto cuenta con 0 para accuracy como IoU. Si solo hubiésemos observado el accuracy global tendríamos un valor de 0.7848, es por ello que tanto la visualización por clases como el uso del IoU (0.0129 medio) son relevantes para la validación total del modelo

Conclusiones

En conclusión, al predecir solo fondo, el modelo carece de utilidad y aunque sigamos entrenando seguirá sin mejorar, por lo que es necesario hacer cambios.

2. Propuestas de mejora

2.1. Modificación de la carga y transformación de datos.

Con el objetivo de poder mejorar los hiperparámetros del modelo, dividimos el dataset de train en: dataset de validación y dataset de train. Además, los dividiremos aleatoriamente, ya que en el dataset original, las imágenes vienen ordenadas.

También aplicaremos transformaciones a los datasets. Sin embargo, para no sesgar los resultados, ciertas transformaciones y cálculos serán realizados solo con el conjunto de train.

En concreto:

- Usamos un DataLoader temporal para obtener estadísticas (media y desviación estándar) para cada canal de las imágenes y posteriormente poder normalizarlas. Este dataloader solo usa el conjunto de train para calcularlas, aunque la normalización se aplicará a todos los conjuntos.
- También usaremos las transformaciones RandomVerticalFlip y RandomHorizontalFlip, como técnicas de Data Augmentation. La idea es aumentar la cantidad de imágenes disponibles para que el modelo generalice mejor y que se adapte mejor a cambios de rotación. Esto solo se realiza para el conjunto de train.

2.2. Modificación de la arquitectura del modelo

Técnicas de regularización

Respecto a la regularización, se ha utilizado la regularización L2, que se implementa mediante el parámetro *weight_decay* del optimizador, que representa el peso que se le da al término regularizador. De forma empírica se ha establecido su valor en 0.0005, lo cual penaliza levemente pesos grandes, ayudando a reducir el sobreajuste.

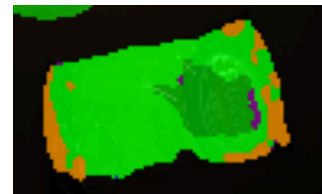
Aumento del número de capas

Aunque la implementación actual del modelo utiliza una profundidad fija de 4 bloques, se consideró la posibilidad de construir una versión más profunda del modelo aprovechando la generación del encoder y el decoder mediante bucles en la clase UNet del cuaderno de Jupyter.

Incrementar la profundidad permitiría capturar características de mayor nivel y mejorar la capacidad del modelo para captar patrones más complejos, especialmente en clases con variabilidad alta o estructuras pequeñas. Sin embargo, una U-Net más profunda implica un coste computacional significativamente mayor (tanto en memoria como en tiempo de entrenamiento), lo que por no exprimir demasiado el cómputo viable, no se ha probado. Sin embargo, creemos que un aumento de la profundidad en la red sería una mejora significativa para el modelo, y consideramos que sería capaz de aumentar el rendimiento, aunque para ello habría que realizar pruebas y observar si el rendimiento añadido compensa el aumento de complejidad en el modelo.

Agrupamiento de las clases en postprocesado

En las predicciones finales observamos que algunos objetos presentan pequeñas zonas de píxeles asignados a una clase distinta de la mayoritaria, ambos pertenecientes claramente al mismo producto. Una posible mejora sería incorporar un procesado posterior al modelo U-Net, cuyo objetivo sería unificar estas regiones para mejorar los resultados. Esto podría hacerse mediante:



- **Filtrado “a mano”** para limpiar estas zonas. La idea sería encontrar píxeles o grupos de píxeles de una clase, que se encuentren rodeados por otra clase.
- **Clustering sobre las predicciones** (k-means o DBSCAN), agrupando píxeles según similitud espacial y de clase para forzar que cada objeto quede predicho de forma homogénea.

Aunque no se ha implementado por razones de tiempo y porque excede de los objetivos de este proyecto (implementar un modelo U-Net), consideramos que la implementación de esta estrategia podría mejorar en gran manera las predicciones de los objetos a nivel general y reduciría de gran manera los errores de los mismos.

2.3. Modificación de la función de coste

Como función de coste alternativa usamos la misma función Cross Entropy pero cambiando el peso de la clase 0 a un valor pequeño (0.2), manteniendo los pesos del resto de clases en 1. El razonamiento detrás de esto es que queremos que predecir correctamente el fondo no valga lo mismo que predecir la clase de un objeto del supermercado, ya que el objetivo es clasificar los objetos, no diferenciar entre fondo y objetos en primer plano.

2.4. Modificaciones del algoritmo de entrenamiento (o del rendimiento del modelo)

Learning rate scheduler

Debido a que durante el entrenamiento las métricas del modelo tenían oscilaciones grandes, bajamos el learning rate de 0.001 a 0.0005 e incluimos el learning rate scheduler *ReduceLROnPlateau*, que reduce el valor del learning rate cuando el criterio ha parado de mejorar durante el número de iteraciones prefijado.

2.5. Otras métricas de rendimiento

Estábamos usando la accuracy para evaluar el modelo, pero como habíamos comentado es un problema desbalanceado y por lo tanto no es la medida que nos parta más información. Para ser capaces de evaluar mejor el modelo vamos a implementar la accuracy por clases y la métrica iou. El accuracy por clases sigue el mismo cálculo que el accuracy pero separada para cada clase de forma que podamos observar del accuracy global que parte corresponde al fondo y cómo de preciso es detectando cada tipo de objeto para poder ver en cuáles falla más y cuáles es más preciso.

El iou (intersection over union) es una métrica típica de segmentación de imagen que mide el solapamiento de la predicción y la máscara entre 0 y 1. Se calcula dividiendo el área de la intersección (aciertos) entre la unión (aciertos + falsos negativos + falsos positivos). Lo calculamos para cada clase partiendo del iou por clase, calcularemos también la media y la media ponderada en función del tamaño de cada clase.

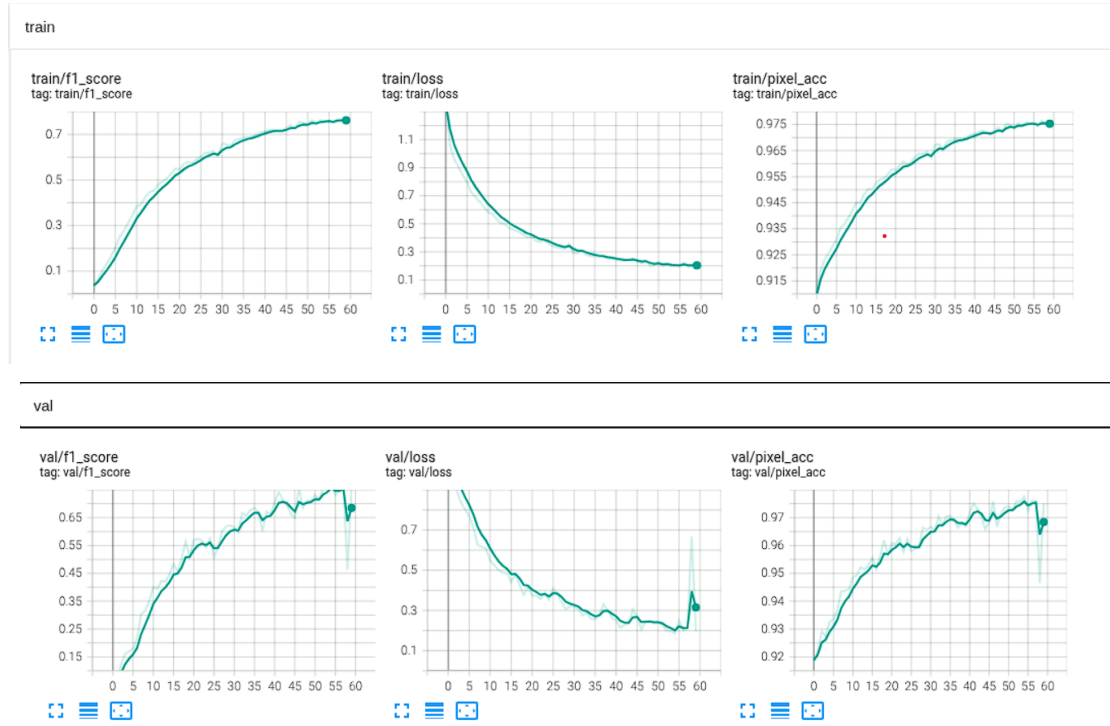
2.6. Resultados modelo mejorado

(*Fase2.ipynb*)

Tras lo comentado anteriormente, el entrenamiento final se hizo con los siguientes hiperparámetros: 60 épocas, *batch_size* = 4, *learning_rate* = 0.0005, *weight_decay* = 0.0005. Para una mayor exploración en los resultados, en la carpeta comprimida adjunta se encuentra el archivo para poder visualizar el Tensorboard completo (*tensorboard.0*). Adicionalmente, en la misma carpeta se incluyen los archivos *model.pth*, *optimizer.pth* y *scheduler.pth* que permiten cargar el modelo directamente para realizar pruebas complementarias o re ejecutar el archivo *Fase2.ipynb*.

A continuación mostraremos salidas de Tensorboard que tienen algo que destacar, pero el fichero con los logs para la visualización está incluido en el archivo .zip de la entrega. Tras finalizar el entrenamiento obtenemos los siguientes resultados:

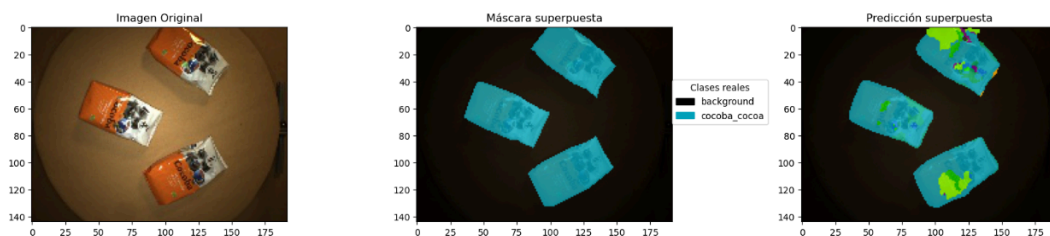
Evolución de la pérdida y accuracy

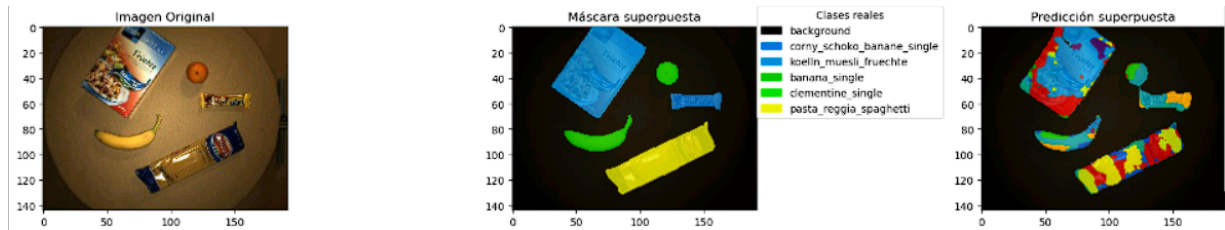


A partir de las salidas de tensorboard podemos observar que el modelo está disminuyendo el loss en train correctamente. Además al llegar a la epoch 60, la pendiente es casi horizontal por lo que podemos dar por finalizado el entrenamiento.

En validación vemos que en alguna de las últimas epochs da un pico en el loss, probablemente por el pequeño *batch_size* usado. Podría ser buena idea entrenarlo alguna epoch más para que vuelva a los valores en los que estaba o volver a entrenarlo con un learning rate ligeramente más pequeño.

Predicciones reales



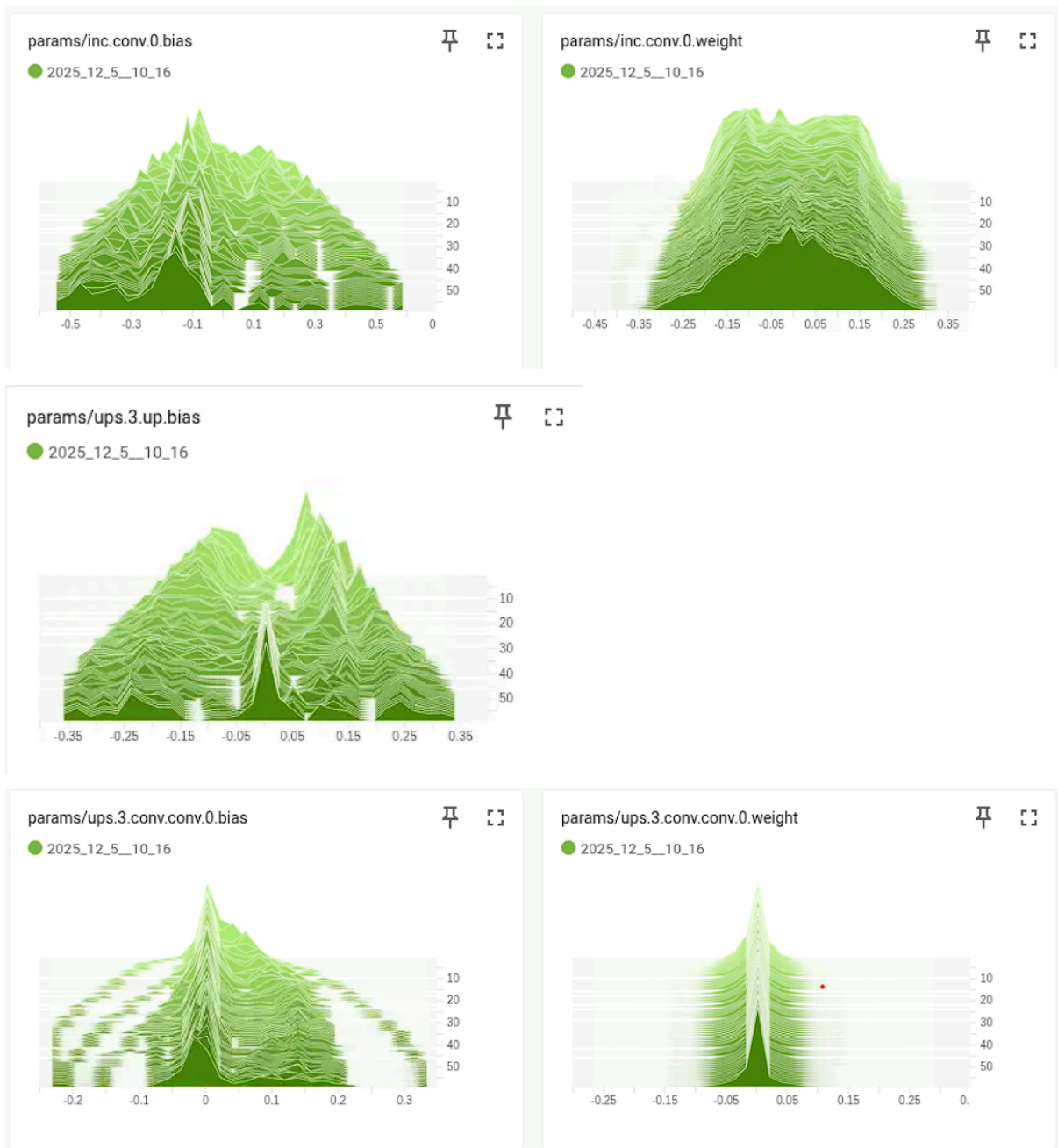


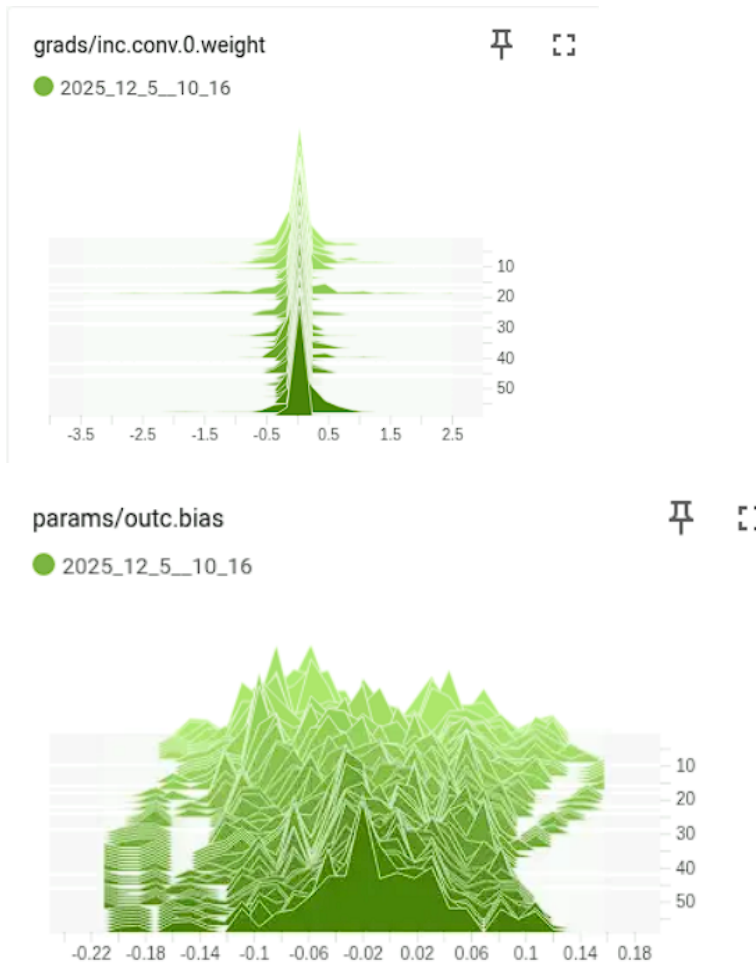
Mapas de probabilidades por clase



Mostramos las tres posibles casuísticas que se pueden dar a la hora de extraer los mapas de probabilidades para cada una de las 60 clases. Para el objeto que ponemos como ejemplo, la clase que le corresponde es la 11, por ello se observa que para esa clase el color amarillo () se superpone en su mayoría sobre el objeto a predecir. Por el contrario, para la clase 0 (fondo) el color amarillo se dispone en el resto de la imagen. Para el resto de clases, que no son la real ni el fondo, no debería verse una gran cantidad de color sobre el objeto a excepción de aquellas clases para las que se pueda predecir erróneamente.

Series temporales de parámetros e histogramas de gradientes





En las gráficas podemos observar la evolución de los parámetros del modelo. Los pesos muestran una distribución correcta, lo que indica que el modelo está aprendiendo. Sin embargo, en los histogramas finales observamos gradientes con magnitudes excesivas y sesgos (bias) con un comportamiento inestable. Esto es un problema y nos indica que para futuros entrenamientos es necesario reducir el learning rate y posiblemente tomar otras medidas.

Rendimiento en el conjunto de test

Tras probar el modelo en el conjunto de test se han obtenido los siguientes resultados, mostramos valores para el conjunto total. Los valores para las distintas clases se encuentran en el archivo que contiene el Tensorboard (*tensorboard.0*)

- **Test Loss:** 2.8607
- **Global Pixel Accuracy:** 0.6747
- **Mean IoU:** 0.1248
- **Weighted IoU:** 0.6327

Para el total del conjunto de test, el Accuracy que se obtiene es del 67,47%, puesto que el fondo presenta un mayor peso no es de extrañar que el valor de WeightedIoU sea alto, similar al de Accuracy, en comparación con el valor de IoU.

En cuanto a los valores para las distintas clases (se visualizan en el archivo *tensorboard.0*) como es de esperar hay clases para las que se obtiene un mejor valor de Accuracy e IoU y otras para las que los valores son bajos o incluso 0. Destacan ciertas clases con un alto valor de Accuracy como la 19 o 32 que superan el 80%.

Conclusiones

En conclusión, hemos conseguido un modelo que ya no predice solo fondo, además de tener unos valores para las métricas evaluadas aceptables. Sin embargo, como comentamos, el modelo admite mejoras en su estructura y en la búsqueda de hiperparámetros si se desea obtener un mejor rendimiento.

Tampoco hay que ser muy críticos con la diferencia entre train/validación con test, ya que el primero conjunto tan solo presenta un fondo y una clase por foto, mientras que en test hay varios fondos y en la misma imagen aparecen varios productos distintos. Por lo tanto la diferencia en este aspecto no se debe solo a sobreajuste o falta de generalización, si no que hay grandes diferencias entre la información con la que se entrena y con la que se evalúa el modelo.